

Why We Need Service Mesh Implementation Patterns

This article looks at the architectural patterns for implementing and deploying application resilience, security and containers using service mesh in microservices architecture.



Many large enterprises have begun modernisation of their applications as part of their digital transformation initiatives. They are packaging applications written in Java stack earlier into container technology, so that these can be run in hybrid multi-cloud environments. While doing so, architects and designers take a conscious decision to separate architectural concerns such as basic security (authentication and authorisation), application resiliency, traffic management and external service connectivity from the functional architecture of the modernised microservices based applications. To take care of these concerns, services are enmeshed within a service mesh. The architectural patterns that follow in this article have been harvested from some real-life implementation projects, where

the service mesh is implemented by Istio and OpenShift.

Application resiliency using circuit breaker and pool ejection

The problem it solves

In distributed microservices architecture, application failures, latency spikes and other network failures have a cascading effect on the services in the call chain and hence in the overall system's resiliency. Microservices written in different languages and deployed in different nodes within and across clusters have different response times and failure rates. Even if one service within a chain of services does not perform well, the entire availability and response time of the system is affected badly.